

ScrollSense — Assignment 1

Ayush Kanojiya DA24B037

Contents

1	Deliverable A — Assumptions and Scope	2
1.1	A.1 — Twelve Numbered Assumptions	2
1.2	A.2 — Three Requirements Being Descoped	4
2	Deliverable B — Conceptual Design	5
2.1	B.1 — ER Diagram	5
2.2	B.2 — Temporal Register	11
2.3	B.3 — The Three Hardest Decisions	12
3	Deliverable C — Logical Design and Relational Algebra	13
3.1	C.1 — Relation Listing	13
3.2	C.2 — Mapping Decision Log	16
3.3	C.3 — Relational Algebra	17
4	Deliverable D — Normalisation	19
4.1	D.1 — Functional Dependency Analysis	19
4.2	D.2 — The Denormalisation Not Done	21
5	Deliverable E — Physical Implementation	23
5.1	E.1 — SQLite DDL	23
5.2	E.2 — Constraint Trace Table	25
5.3	E.3 — Data Generator	27
5.4	E.4 — JSON Use	28
6	Deliverable F — Queries	30
7	Deliverable G — Views and Transactions	36
7.1	G.1 — Views	36
7.2	G.2 — Written Answers	36
7.3	G.3 — Transaction Scripts	37
	Bonus	39
	Appendix — LLM Assistant Usage	41

1 Deliverable A — Assumptions and Scope

1.1 A.1 — Twelve Numbered Assumptions

#	Where	Ambiguity (as a question)	My assumption	Why	What breaks if reversed
A1	§2.2	Can one video carry more than one audio track?	Exactly zero or one	§2.2 describes “an optional audio track” (singular); feed UI plays a single track	Video–Track becomes M:N, needs a junction table; “50,000 clips use one trending sound” becomes ambiguous about direction
A2	§2.4	Can one impression yield more than one <i>view</i> ?	No — an impression yields zero or exactly one view; multiple viewing segments belong to that one view	§2.4: “One impression can yield several viewing segments,” strictly separating impression → view → segments as a hierarchy, not view → views	Multiple independent views per impression would double-count watch time and completion across views instead of aggregating within one view’s segments
A3	§2.4	Is an impression with no resulting view a stored row, or an absence?	A stored row — impressions are logged unconditionally at viewport entry	§2.4 records impressions independently of the 300ms threshold; §2.6 requires reports correct for “shown 900 times, watched twice”	If not stored, funnel/completion reporting cannot distinguish “never shown” from “shown but not watched,” which §2.6 forbids
A4	§2.5	Can a tool call belong to more than one turn?	No — a tool call belongs to exactly one turn; nesting is within that turn’s call tree	§2.5 describes tool calls as issued by the assistant while answering <i>one</i> turn; nesting is a tree, not a shared resource	Cross-turn tool calls would need an M:N join and break the tree recursion that F11 assumes
A5	§2.5	Does a recommendation need its own row separate from a generic impression?	Yes — (session, turn, position, clip) is a distinct entity linked to an impression	§2.5’s “critical link” traces “clip 8821 at position 2 of turn 3 in session 4471” — this needs a first-class key, not inference	Folding recommendations into ordinary impressions turns F13 into a fragile heuristic join instead of a direct FK path
A6	§2.1	Does handle uniqueness apply globally or only among active accounts?	Only among active accounts; a deactivated/deleted account’s handle can be reused	§2.1 states “unique across active accounts” explicitly, and separately allows deactivated/pending-deletion states	Global-forever uniqueness would permanently lock a released handle, and a plain UNIQUE constraint cannot express “among active” at all
A7	§2.7	Is “current state” a separately stored, overwritten column?	No — current state is <i>derived</i> (e.g. latest row of an append-only history); there is only one write, not two	§2.7 diagnoses the failure as exactly the two-write pattern and hints “a better answer... older than they are”	A separate stored column keeps the described failure mode possible even inside transactions unless carefully synchronised

#	Where	Ambiguity (as a question)	My assumption	Why	What breaks if reversed
A8	§2.2	Is a hashtag a materialised entity, or text parsed at query time?	Not materialised — hashtags stay embedded in caption text; extraction happens at query time	§2.2: “Hashtags are not a separate field... they live inside the caption text”; F5 explicitly tests parsing via <code>lower()</code> , <code>trim()</code> , <code>replace()</code> , <code>instr()</code> , <code>LIKE</code>	A hashtag table would make F5 a trivial join and remove the case/whitespace/Tamil edge cases F5 is designed to surface
A9	§2.3	When a block breaks a follow, is the follow deleted or ended?	Retained with an end marker; follows are never hard-deleted	§3’s mandate to reconstruct past state as of any timestamp argues against destructive deletes of historical facts	Hard deletion would make “was X following Y at time T” unanswerable once a later block occurred — exactly the leak §3 warns against
A10	§2.2	Does a later moderation decision correct or supersede an earlier one?	Supersede — every decision (automated or human) is an immutable, timestamped, append-only fact	§2.2: “states change over the video’s life” implies a sequence of facts, not a mutable field with an audit trail bolted on	Mutating/deleting earlier decisions makes F8 (states changed >2 times, full chronological sequence) unanswerable
A11	§2.4	At what grain does watch time accumulate?	Per view-segment, aggregated upward as needed; never stored as a running total on a coarser entity	§2.4 ties watch time explicitly to segments (scroll-away-and-return, looping); the segment is the only level at which one number is unambiguous	A running total on (user, video) makes “shown 900 times, watched twice” (§2.6) unverifiable and becomes exactly the D.2 denormalisation risk
A12	§2.5	Is model pricing a validity-interval table, or a current-price column?	A temporal (validity-interval) table: (model, rate_type, valid_from, valid_to, rate), joined by the turn’s timestamp	§2.5 states plainly “a price change must never alter last quarter’s reported costs” and flags “this constraint has a clear schema consequence”	A current-price column makes <code>v.turn_cost</code> (G.1) impossible to build as specified, and historical costs would silently drift on every rate change

Note on selection

These twelve are drawn across §§2.1–2.7, not clustered in §2.1–2.2. Each reversal column names a structural schema consequence (a new junction table, a broken query, an unenforceable constraint) rather than a mere type change, per the instruction to hunt cardinality and grain ambiguities first. A2, A7, A9, A10, A11 and A12 are mutually reinforcing: A7’s rejection of a duplicated current-state column is what makes A9/A10’s append-only design coherent, and A12’s temporal pricing is what makes A11’s segment-level watch time reportable without drift.

1.2 A.2 — Three Requirements Being Descoped

What I am not building	What the brief seemed to ask for	Why the cost exceeds the value (Module 1)	Re-entry path
Full rubric (separate dimensions for helpfulness/groundedness/safety as distinct queryable sub-entities, with per-dimension thresholds and rubric versioning)	§2.5: “An LLM judge scores helpfulness, groundedness and safety”	Module 1 targets relational modelling and grain, not ML-evaluation framework design; three correlated scores per (turn, judge-run) add width, not new relational structure; F14 needs only a single threshold comparison	Add <code>JudgeScoreDimension(...)</code> once real rubric-versioning is required; current single-score design absorbs it by widening
Multi-region/locale infrastructure for audio licensing (region-specific rights and region-specific trending as stored state rather than derived)	§2.5 mentions <code>fetch_trending_audio(region)</code> as a tool-call argument, implying regional variation	Trending appears only as a tool-call argument, never as a business entity in §2.2’s licensing description; building a full rights-by-region schema invents a requirement not in the brief	Add <code>TrackRegionRights(...)</code> if licensing-by-region becomes a real requirement; trending remains a query, not stored state
Real-time / full ML model lineage tracking for the ranking model (artifact registry, training-snapshot linkage, deployment-window guarantees)	§2.4 requires “the model version that ranked it there” per impression, extensible into full lineage tracking	The brief asks only that the version <i>string</i> be recorded per impression; full artifact/training lineage is an MLOps concern outside a relational-schema assignment, and §3’s reconstruction requirement is satisfied by the version label alone	Add <code>ModelVersion(...)</code> later; the impression’s <code>model_version</code> column becomes an FK into it — a non-breaking migration

2 Deliverable B — Conceptual Design

2.1 B.1 — ER Diagram

Legend



Crow's-foot notation: a bar = exactly one, a circle = zero (optional), a crow's foot = many. Read the mark closest to an entity as that entity's own participation constraint on the relationship.

Step 1 — Entity Inventory

Entity	One row = ...	Identifier	Kind
AppUser	one registered account	user_id	strong
UserInterestDeclared	one category a user explicitly chose at signup	(user_id, category)	associative
UserInterestInferred	one ML-assigned interest score for a user, as of one weekly refresh	(user_id, category, refreshed_at)	event
InterestSuppression	one user's decision to suppress one inferred category	(user_id, category)	weak
CreatorTierPeriod	one continuous interval during which a creator held one monetisation tier	(creator_id, valid_from)	temporal
Follow	one user following another, from start until (optionally) ended	(follower_id, followee_id, started_at)	weak
Block	one user blocking another, at one instant	(blocker_id, blocked_id, started_at)	weak
Mute	one user muting another, at one instant	(muter_id, muted_id, started_at)	weak
Video	one uploaded clip	video_id	strong
AudioTrack	one audio track, original or catalogue-licensed	track_id	strong
ModerationDecision	one moderation verdict on one video, by one classifier or reviewer, at one instant	(video_id, decided_at)	event
Impression	one clip entering one user's viewport at one instant	(user_id, video_id, ts)	event
View	one 300ms-threshold-crossing playback episode arising from one impression	view_id	weak
ViewSegment	one continuous stretch of playback within one view	(view_id, segment_seq)	weak
EngagementSignal	one explicit user action (like/save/share/comment/follow-from-feed/not-interested/report) on one video	signal_id	event
LikeRetraction	one instance of a user retracting a prior like, as its own negative signal	retraction_id	event
AgentSession	one user's conversation with the agent	session_id	strong
AgentTurn	one user-message-plus-assistant-response pair within one session	turn_id	weak
ToolCall	one tool invocation issued while answering one turn, possibly nested inside another	tool_call_id	weak

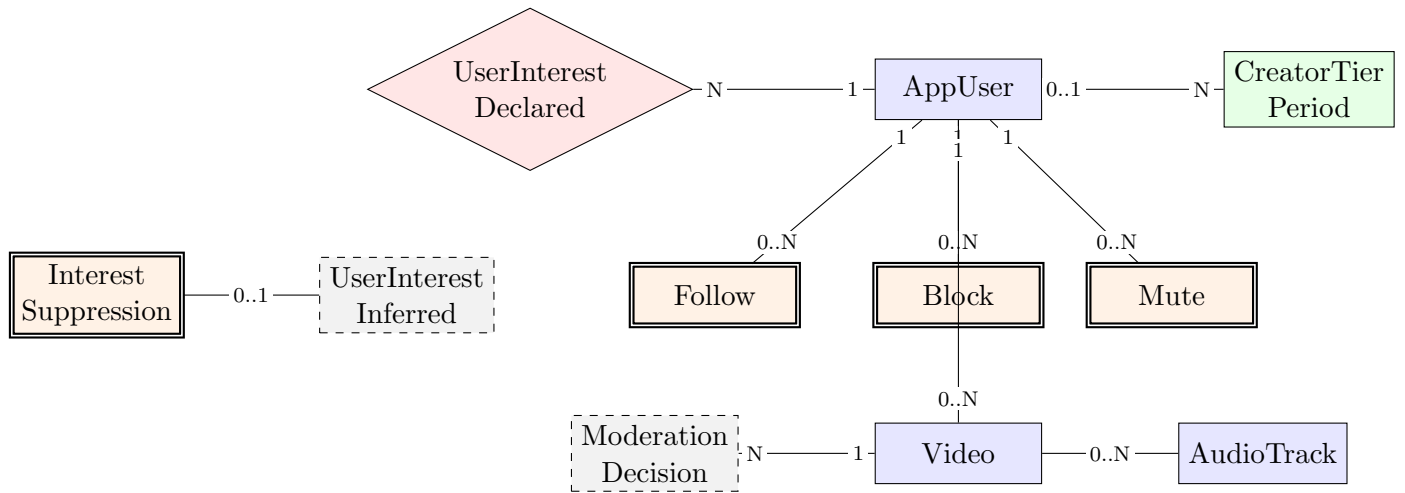
Entity	One row = ...	Identifier	Kind
TurnUsage	one turn's token counts, priced against one model-pricing period	turn_id	weak
ModelPricingPeriod	one interval during which one model's input/output/cached rates held	(model_id, rate_type, valid_from)	temporal
PromptTemplateVersion	one exact version of one prompt template's text, as of one edit	(template_id, version)	temporal
Recommendation	one clip placed at one shelf position within one turn	(turn_id, position)	associative
JudgeScore	one LLM-judge scoring run (helpfulness, groundedness, safety) for one turn	(turn_id, judged_at)	event
UserRating	one thumbs up/down a user gave one turn's explanation	(turn_id, rated_at)	weak

Step 2 — Relationship Inventory

Relationship	From → To	Cardinality	Participation	Attributes on relationship
declares	AppUser → UserInterestDeclared	1 : 0..N	user optional, category mandatory	declared_at
holds tier	AppUser → CreatorTierPeriod	1 : 0..N	user optional (non-creators: 0), period mandatory	valid_from, valid_to
follows	AppUser → AppUser (via Follow)	0..N : 0..N	both optional	started_at, ended_at
blocks	AppUser → AppUser (via Block)	0..N : 0..N	both optional	started_at
mutes	AppUser → AppUser (via Mute)	0..N : 0..N	both optional	started_at
infers	AppUser → UserInterestInferred	1 : 0..N	user optional, row mandatory	confidence, refreshed_at
suppresses	InterestSuppression → UserInterestInferred	0..1 : 1	suppression optional	suppressed_at
owns	AppUser → Video	1 : 0..N	user optional, video mandatory	—
uses	Video → AudioTrack	0..N : 0..N	video optional, track optional	—
decided on	ModerationDecision → Video	N : 1	decision mandatory, video optional	decided_at, decided_by
impressed	AppUser → Impression	1 : 0..N	user mandatory, impression optional	—
impressed	Video → Impression	1 : 0..N	video mandatory, impression optional	feed_position, model_version
yields	Impression → View	1 : 0..1	impression mandatory, view optional	—
segments	View → ViewSegment	1 : 1..N	view mandatory, segment mandatory	watch_ms, completed
signals	AppUser → EngagementSignal	1 : 0..N	user mandatory, signal optional	—
signals	Video → EngagementSignal	1 : 0..N	video mandatory, signal optional	signal_type, ts
retracts	EngagementSignal → LikeRetraction	0..1 : 1	original like optional (only likes retract), retraction mandatory	retracted_at
contains	AgentSession → AgentTurn	1 : 1..N	session mandatory, turn mandatory	seq

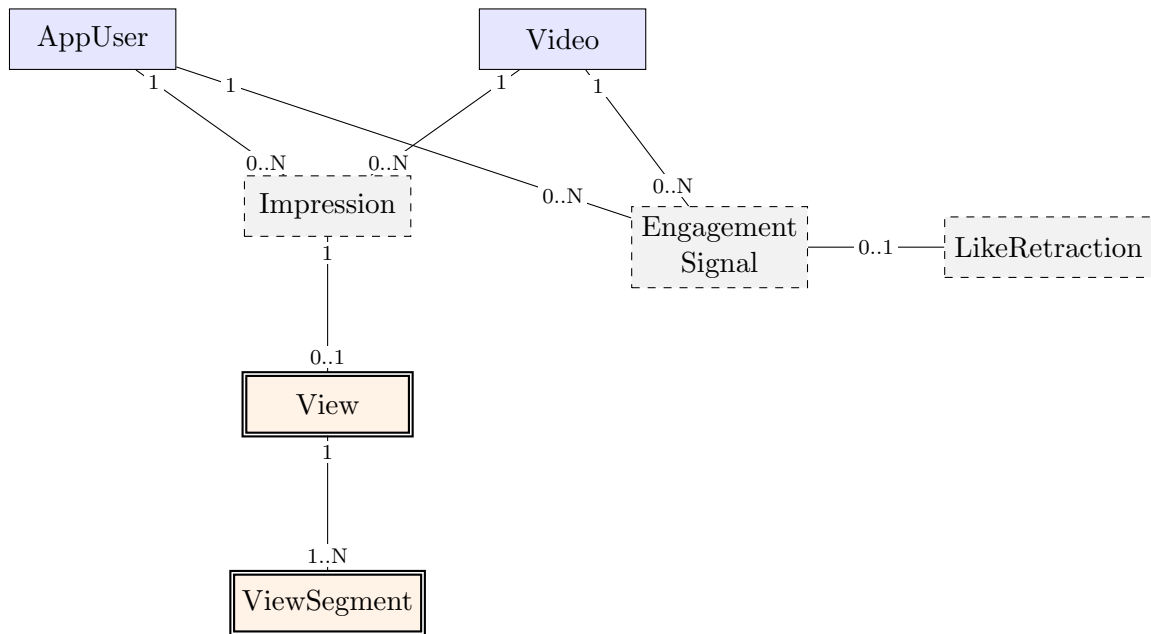
Relationship	From → To	Cardinality	Participation	Attributes on relationship
issues	AgentTurn → ToolCall	1 : 0..N	turn mandatory, call optional	—
nests	ToolCall → ToolCall	0..1 : 0..N	child optional, parent optional	—
meters	ToolCall → TurnUsage	1 : 1	call mandatory, usage mandatory	—
priced at	TurnUsage → ModelPricingPeriod	1 : 1	usage mandatory, pricing mandatory	—
generated by	AgentTurn → PromptTemplateVersion	N : 1	turn mandatory, template mandatory	—
shelves	AgentTurn → Recommendation	1 : 0..N	turn mandatory, recommendation optional	position
resolves to	Recommendation → Impression	0..1 : —	recommendation optional (not all become impressions)	—
judged by	AgentTurn → JudgeScore	1 : 0..1	turn mandatory, score optional	helpfulness, groundedness, safety
rated by	AgentTurn → UserRating	1 : 0..1	turn mandatory, rating optional	thumbs_up

Diagram 1 — Identity, Content and Social Graph



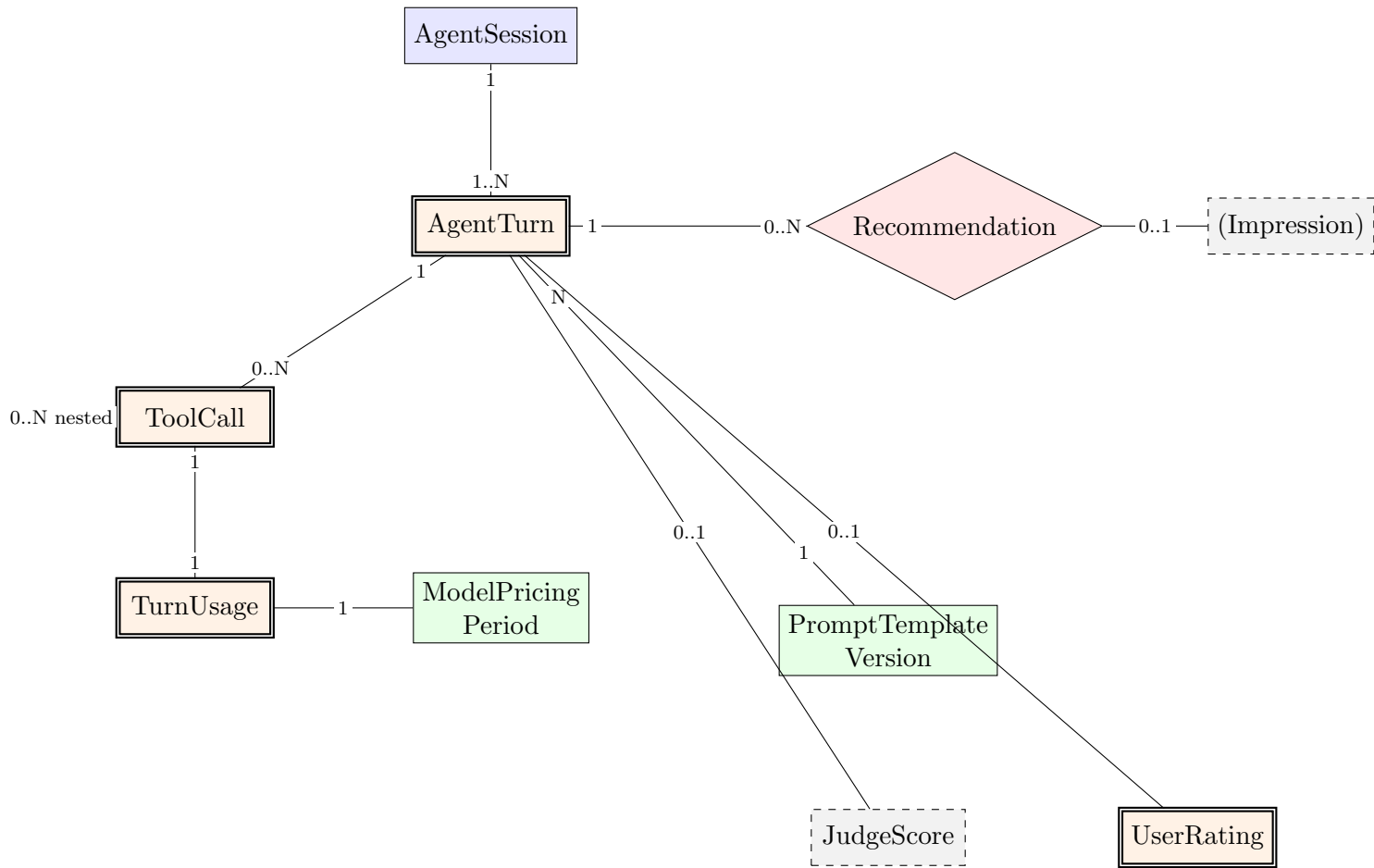
Reading notes: *InterestSuppression* is weak on (user, category) from *UserInterestInferred* (A-suppression survives refresh, per §2.1). *Follow/Block/Mute* are all self-referencing on *AppUser* and modelled as weak entities carrying their own timestamps rather than as plain attributes, since a block must be able to look up and end an existing follow (A9). *ModerationDecision* is append-only (A10); *Video*’s “current state” is a derived view (B.2), not a stored column (A7). *CreatorTierPeriod* is a validity-interval temporal entity (Finance’s “tier last March” requirement).

Diagram 2 — Watch Telemetry



Reading notes: this is the strict hierarchy from A2/A3 — Impression is logged unconditionally at viewport entry; a View exists only if the 300ms threshold was crossed (0..1, not 1..1); ViewSegment is where watch time actually accumulates (A11), never as a running total on Impression or Video. LikeRetraction is a separate event row, not a delete of the original like (per §2.4's explicit instruction to model retraction as its own negative signal).

Diagram 3 — Agent Layer



Reading notes: ToolCall self-references for arbitrary-depth nesting (A4), all within one turn's tree. PromptTemplateVersion and ModelPricingPeriod are both temporal (A12) so that historical turns keep the exact template text and price in force at the time. Recommendation is the associative entity implementing the "critical link" (A5): it connects a turn's shelf position to the Impression that resulted, letting F13 trace clip → position → watch outcome directly.

2.2 B.2 — Temporal Register

Entity	Overwrite	Validity intervals	Append-only log	Justification
Creator monetisation tier		✓		Finance must answer “which tier last March,” an as-of-T query over a continuous interval, not a point event (A — CreatorTierPeriod, §2.2).
Video moderation state			✓	Current state must be derivable without a second write (A7); an immutable decision history lets F8’s full transition sequence be reconstructed exactly (A10).
Inferred user interests			✓	Weekly refreshes are discrete recomputation events, not a continuously valid interval; suppression (A1) must survive future refreshes, which only works if past inferred rows are never overwritten.
Likes and retractions			✓	A retraction is a negative signal in its own right, not a state flip on the like row (§2.4, explicit instruction); both the like and its retraction must remain queryable as distinct historical facts.
Prompt template text		✓		Edits happen several times a week and every historical turn must stay linked to the exact text in force when it ran (§2.5); this is a validity interval on (template_id, version), not a log of edits.
Model token pricing		✓		§2.5 states plainly a price change must never alter last quarter’s reported cost; <code>v_turn_cost</code> (G.1) joins a turn’s timestamp against the rate interval in force at that instant (A12).
Account state (active / deactivated / pending-deletion)		✓		The 30-day recovery window is itself an interval (deletion requested → recovery deadline); overwriting a single <code>status</code> column would lose the deadline needed to decide restorability.
Follow relationship		✓		A block must end an existing follow without deleting it (A9), and §3 requires reconstructing “was X following Y at T”; <code>started_at/ended_at</code> on the Follow row is the natural validity interval.
Handle (username)	✓			Changeable at most twice a year and only the current value is ever queried by feed/profile lookups; history of past handles is not required by any stated consumer (§2.6), so overwrite is the cheaper correct choice.

2.3 B.3 — The Three Hardest Decisions

1. Whether “current state” should be a column at all

What I built: Video’s moderation state is never stored directly. It is derived — the latest row of the append-only `ModerationDecision` log, exposed via a view (`v_video_current_state`, G.1) that resolves to the most recent `decided_at` per video.

What I rejected: A `current_state` column on `Video`, updated alongside each new decision row — the obvious first instinct, since Trust & Safety wants state “now, in one lookup” (§2.6).

Why I rejected it: §2.7 names this exact pattern as the founders’ bug: two writes for one fact, and roughly once a week one lands without the other. A stored column reintroduces that failure inside my own schema unless every future write path remembers to keep both in sync. A derived view can’t drift from its source, because there is only one write. The cost is a small read-time scan instead of a column read — cheap at this scale, and if it stopped being cheap, only the view body would need to change, not every consumer of it.

2. Whether a block-ended follow is deleted or merely ended

What I built: Follow carries `started_at` and a nullable `ended_at`. A block ends any existing follow in both directions by setting `ended_at`, never by deleting the row.

What I rejected: Hard-deleting the Follow row on block — the more natural first reading of “a block must break any existing follow,” and simpler to write.

Why I rejected it: §3 requires reconstructing state “as it was at any past timestamp.” A hard delete makes “was X following Y last Tuesday” unanswerable the instant a later block occurs — the historical fact is gone, not archived. This matters concretely: a training row for an impression shown before the block must still see the follow relationship that was true then, even after the block has since fired. A soft end-marker preserves that; a delete does not.

3. Whether engagement signals are one polymorphic table or seven

What I built: A single `EngagementSignal` entity with a `signal_type` discriminator (`like`, `save`, `share`, `comment`, `follow_from_feed`, `not_interested`, `report`), plus a nullable `share_destination` meaningful only when `signal_type` = ‘share’.

What I rejected: Seven separate tables, one per signal, each with only the attributes it needs and no nullable columns for attributes that don’t apply.

Why I rejected it: §2.4/§2.6 both treat these seven as one funnel stage — “most views produce no explicit signal,” completion-rate reporting that must not drop creators, and F10’s `UNION` vs `UNION ALL` comparison, which only makes sense if the signals share one queryable shape. Seven tables would force every funnel query into a seven-way union, and each new signal type later would mean a new table, not one new `CHECK` value. The cost — `share_destination` is `NULL` for six of seven types — is documented in E.1’s `NOT NULL` audit as “does not apply,” distinct from “not yet known,” and is smaller than fragmenting the entity every funnel query needs uniformly.

A fourth close call, noted but not elaborated: Recommendation→Impression is modelled as partial (0..1), since a recommended clip the user never scrolls to never produces an impression — the two entities’ existence is genuinely decoupled.

3 Deliverable C — Logical Design and Relational Algebra

3.1 C.1 — Relation Listing

```
AppUser(  
  user_id      : bigint          -- PK  
  phone        : text NULL  
  google_id    : text NULL  
  handle       : varchar(30)  
  display_name : varchar(60)  
  account_state : text          -- 'active'|'deactivated'|'pending_deletion'  
  created_at   : timestampz  
)  
PK: user_id  
  
UserInterestDeclared(  
  user_id      : bigint          -- FK -> AppUser  
  category     : text  
  declared_at  : timestampz  
)  
PK: (user_id, category)  
FK: user_id -> AppUser  
  
UserInterestInferred(  
  user_id      : bigint          -- FK -> AppUser  
  category     : text  
  refreshed_at : timestampz  
  confidence   : real           -- 0.0 .. 1.0  
)  
PK: (user_id, category, refreshed_at)  
FK: user_id -> AppUser  
  
InterestSuppression(  
  user_id      : bigint          -- FK -> AppUser  
  category     : text  
  suppressed_at : timestampz  
)  
PK: (user_id, category)  
FK: user_id -> AppUser  
  
CreatorTierPeriod(  
  creator_id   : bigint          -- FK -> AppUser  
  tier         : text  
  valid_from   : timestampz  
  valid_to     : timestampz NULL -- NULL = current  
)  
PK: (creator_id, valid_from)  
FK: creator_id -> AppUser  
  
Follow(  
  follower_id  : bigint          -- FK -> AppUser  
  followee_id  : bigint          -- FK -> AppUser  
  started_at   : timestampz  
  ended_at     : timestampz NULL  
)  
PK: (follower_id, followee_id, started_at)  
FK: follower_id -> AppUser ; followee_id -> AppUser  
  
Block(  
  blocker_id   : bigint          -- FK -> AppUser  
  blocked_id   : bigint          -- FK -> AppUser  
  started_at   : timestampz  
)
```

```

PK: (blocker_id, blocked_id, started_at)
FK: blocker_id -> AppUser ; blocked_id -> AppUser

Mute(
    muter_id : bigint          -- FK -> AppUser
    muted_id : bigint          -- FK -> AppUser
    started_at : timestampz
)
PK: (muter_id, muted_id, started_at)
FK: muter_id -> AppUser ; muted_id -> AppUser

AudioTrack(
    track_id      : bigint          -- PK
    origin_video_id : bigint NULL    -- FK -> Video, self-referencing pair
    source         : text           -- 'original' | 'licensed'
    license_ref    : text NULL
)
PK: track_id

Video(
    video_id      : bigint          -- PK
    owner_id      : bigint          -- FK -> AppUser
    duration_ms   : integer         -- CHECK BETWEEN 20000 AND 90000
    caption       : varchar(2200)
    audio_track_id : bigint NULL     -- FK -> AudioTrack
    uploaded_at   : timestampz
)
PK: video_id
FK: owner_id -> AppUser ; audio_track_id -> AudioTrack

ModerationDecision(
    video_id      : bigint          -- FK -> Video
    decided_at    : timestampz
    state         : text           -- 'pending' | 'live' | 'age_restricted' | 'demoted' | 'taken_down'
    decided_by_type : text         -- 'automated' | 'human'
    decided_by_id  : bigint NULL
)
PK: (video_id, decided_at)
FK: video_id -> Video

Impression(
    impression_id : bigint          -- PK
    user_id       : bigint          -- FK -> AppUser
    video_id      : bigint          -- FK -> Video
    ts           : timestampz
    feed_position : integer
    model_version : text
)
PK: impression_id
FK: user_id -> AppUser ; video_id -> Video

View(
    view_id      : bigint          -- PK
    impression_id : bigint          -- FK -> Impression, UNIQUE (0..1)
    started_at    : timestampz
)
PK: view_id
FK: impression_id -> Impression (UNIQUE)

ViewSegment(
    view_id      : bigint          -- FK -> View
    segment_seq  : integer
    watch_ms     : integer

```

```

    completed      : integer          -- 0/1
)
PK: (view_id, segment_seq)
FK: view_id -> View

EngagementSignal(
    signal_id      : bigint           -- PK
    user_id        : bigint           -- FK -> AppUser
    video_id       : bigint           -- FK -> Video
    signal_type    : text             -- 'like'|'save'|'share'|'comment'
                                     -- '|follow_from_feed'|'not_interested'|'report'
    ts             : timestampz
    share_destination: text NULL      -- meaningful only if signal_type='share'
    comment_text   : text NULL       -- meaningful only if signal_type='comment'
)
PK: signal_id
FK: user_id -> AppUser ; video_id -> Video

LikeRetraction(
    retraction_id  : bigint           -- PK
    like_signal_id  : bigint           -- FK -> EngagementSignal, UNIQUE
    retracted_at   : timestampz
)
PK: retraction_id
FK: like_signal_id -> EngagementSignal (UNIQUE)

AgentSession(
    session_id     : bigint           -- PK
    user_id        : bigint           -- FK -> AppUser
    started_at     : timestampz
)
PK: session_id
FK: user_id -> AppUser

AgentTurn(
    turn_id        : bigint           -- PK
    session_id     : bigint           -- FK -> AgentSession
    seq            : integer
    user_message   : text
    assistant_message : text
    template_id    : bigint           -- FK -> PromptTemplateVersion (composite)
    template_version : integer        -- FK -> PromptTemplateVersion (composite)
    model_id       : text
    temperature    : real
    created_at     : timestampz
)
PK: turn_id
FK: session_id -> AgentSession ; (template_id, template_version) -> PromptTemplateVersion

ToolCall(
    tool_call_id   : bigint           -- PK
    turn_id        : bigint           -- FK -> AgentTurn
    parent_tool_call_id : bigint NULL -- FK -> ToolCall (self)
    name           : text
    arguments_json  : text            -- CHECK (json_valid(arguments_json))
    result         : text NULL
    latency_ms     : integer
    errored        : integer          -- 0/1
)
PK: tool_call_id
FK: turn_id -> AgentTurn ; parent_tool_call_id -> ToolCall

TurnUsage(

```

```

turn_id      : bigint          -- PK, FK -> AgentTurn
model_id     : text
priced_at    : timestampz
input_tokens : integer
output_tokens : integer
cached_tokens : integer
)
PK: turn_id
FK: turn_id -> AgentTurn ; (model_id, priced_at) -> ModelPricingPeriod (resolved by app/view, not FK)

ModelPricingPeriod(
  model_id : text
  rate_type : text          -- 'input'|'output'|'cached_input'
  valid_from : timestampz
  valid_to   : timestampz NULL
  rate       : real
)
PK: (model_id, rate_type, valid_from)

PromptTemplateVersion(
  template_id : bigint
  version      : integer
  valid_from   : timestampz
  valid_to     : timestampz NULL
  text         : text
)
PK: (template_id, version)

Recommendation(
  turn_id      : bigint          -- FK -> AgentTurn
  position     : integer
  video_id     : bigint          -- FK -> Video
  impression_id : bigint NULL    -- FK -> Impression, UNIQUE if present
)
PK: (turn_id, position)
FK: turn_id -> AgentTurn ; video_id -> Video ; impression_id -> Impression (UNIQUE)

JudgeScore(
  turn_id      : bigint          -- FK -> AgentTurn
  judged_at    : timestampz
  helpfulness  : integer
  groundedness : integer
  safety       : integer
)
PK: (turn_id, judged_at)
FK: turn_id -> AgentTurn

UserRating(
  turn_id      : bigint          -- PK, FK -> AgentTurn
  rated_at     : timestampz
  thumbs_up    : integer          -- 0/1
)
PK: turn_id
FK: turn_id -> AgentTurn

```

3.2 C.2 — Mapping Decision Log

ER construct	Mapping applied	Alternative rejected	re-	Why
Follow/Block/Mute (weak, self-referencing on AppUser)	three tables, each with a composite PK (pair of user FKs + <code>started_at</code>)	a <code>RelationshipEvent</code> table with a kind discriminator	single	the three have materially different attribute needs (Follow alone carries <code>ended_at</code> ; Block/Mute don't) and different query patterns; a shared discriminator table would need nullable columns for the case that doesn't apply, same objection as <code>EngagementSignal</code> but here the shapes diverge more than they overlap
ToolCall (A4)	self-nesting one table with nullable <code>parent_tool_call_id</code> FK to itself	a separate <code>ToolCallEdge</code> junction table		nesting is a strict tree (A4: one parent per call), so a self-FK is a lossless 1:N mapping; a junction table would be needed only for M:N, which this isn't, and F11's recursive CTE walks a self-FK directly
Recommendation (associative entity, A5)	its own table, PK (<code>turn_id</code> , <code>position</code>), with an optional FK to Impression	folding into Impression via a nullable <code>recommended_position</code> column on Impression		Impression already has a distinct identity from viewport entries that aren't recommendations (organic feed scroll); overloading it would make every impression query filter on whether the column is null, and Recommendation must exist even before any impression results (0..1, not 1..1)
CreatorTierPeriod / ModelPricingPeriod / PromptTemplateVersion (temporal, B.2)	validity-interval tables: (<code>..._id</code> , <code>valid_from</code>) as PK, nullable <code>valid_to</code>	a single current-value column per parent table		B.2 already establishes these as interval strategy; folding into the parent would make <code>v_turn_cost</code> and "tier last March" unbuildable, and would collapse three independent version histories into fields that overwrite
EngagementSignal (multivalued signal type, §2.4)	one table with a <code>signal_type</code> CHECK-constrained column and two nullable type-specific attributes	seven separate tables (Like, Save, Share, ...)		justified at length in B.3(3); funnel queries (F10) need one shared shape, and a discriminator column costs two harmless NULLs rather than a seven-way UNION on every funnel query
ModerationDecision (append-only, A10/B.2)	one growing table, PK (<code>video_id</code> , <code>decided_at</code>), no update path in the application layer	a mutable <code>state</code> column plus a separate audit-log table written alongside it		this is precisely the two-write pattern §2.7 diagnoses as the founders' bug; one append-only table with a derived-view current-state read (B.3(1)) removes the second write entirely

3.3 C.3 — Relational Algebra

F2 — Watch hours and completion rate per creator, live clips only

Let $Live = \sigma_{state='live'}(v_video_current_state) \bowtie Video$

$$F2 = \gamma_{owner_id; SUM/watch_ms/3600000 \rightarrow hours, AVG/completed \rightarrow compl.}(Live \bowtie video_id (Impression \bowtie View \bowtie ViewSegment))$$

The outer join $()$ preserves creators/live clips with zero impressions or zero views; SUM/AVG over the resulting NULLs are handled with COALESCE at the SQL layer, not by the algebra itself.

F3 — Videos with no audio track

$$F3 = Video \bowtie \pi_{video_id}(\sigma_{audio_track_id \text{ IS NOT NULL}}(Video))$$

i.e. an anti-join of Video against itself restricted to rows with a non-null track — the RA equivalent of NOT EXISTS. The NOT IN version differs operationally (not algebraically) only in how SQL evaluates the subquery when it can return NULL; this is why the two row counts can diverge in SQL despite both intending this same anti-join.

F12 — Recommended clips watched to completion, with shelf position

$$F12 = \pi_{session_id, position}(AgentSession \bowtie AgentTurn \bowtie Recommendation \bowtie_{impression_id} Impression \bowtie View \bowtie_{\sigma_{completed=1}}(ViewSegment))$$

Equivalence-preserving rewrite. Take F2 and push the selection $\sigma_{state='live'}$ below the join with the watch-telemetry chain rather than filtering after:

Before:

$$\gamma_{owner_id; \dots}(\sigma_{state='live'}(v_video_current_state \bowtie Video \ (Impression \bowtie View \bowtie ViewSegment)))$$

After:

$$\gamma_{owner_id; \dots}((\sigma_{state='live'}(v_video_current_state) \bowtie Video) \ (Impression \bowtie View \bowtie ViewSegment))$$

Why: ViewSegment is the highest-volume table in the schema (one row per continuous playback stretch, across 300k+ impressions). Filtering Video down to live clips *before* the outer join means the join's left-hand input shrinks to only live videos, so the outer join and every downstream aggregate scan fewer rows; filtering after the join means the full watch-telemetry chain is joined and materialised for every video — including non-live ones — only to be discarded afterward.

4 Deliverable D — Normalisation

4.1 D.1 — Functional Dependency Analysis

Relation 1: TurnUsage (agent layer)

RELATION: TurnUsage(turn_id, model_id, priced_at, input_tokens, output_tokens, cached_tokens)

FDs claimed:

F1: turn_id → model_id, priced_at, input_tokens, output_tokens, cached_tokens
(holds because: one turn is answered once, by one model, at one instant -- AgentTurn.model_id and the pricing snapshot are fixed the moment the assistant message is produced)

Closures:

{turn_id}⁺ = {turn_id, model_id, priced_at, input_tokens, output_tokens, cached_tokens} -- all attributes

Candidate keys: {turn_id} (only key; a single-attribute set whose closure is already all attributes has no proper subset to test)

Verdict: BCNF. The only FD is turn_id → everything, and turn_id is a superkey, so no FD violates BCNF. No decomposition needed.

Relation 2: ModelPricingPeriod (temporal)

RELATION: ModelPricingPeriod(model_id, rate_type, valid_from, valid_to, rate)

FDs claimed:

F1: model_id, rate_type, valid_from → valid_to, rate
(holds because: a vendor's rate for one model and one rate_type is fixed for the interval starting at valid_from -- this is the definition of a validity-interval table, B.2)

Closures:

{model_id, rate_type, valid_from}⁺ = all attributes

Candidate keys: {model_id, rate_type, valid_from} (only key)

Note on a rejected second key: valid_to is NULL for whichever period is currently active, so {model_id, rate_type, valid_to} CANNOT be a candidate key -- a key must uniquely determine every row, and every model/rate_type's current period shares valid_to = NULL simultaneously with any other model/rate_type's current period, and NULL is not comparable via = at all. A functional dependency with valid_to on the left therefore does not hold in the presence of open intervals. This is worth stating explicitly rather than silently omitting it, since the natural first instinct (an interval's end should identify it as well as its start) is wrong exactly because of the open-interval case.

Verdict: BCNF. The one FD's left side, {model_id, rate_type, valid_from}, is a superkey (its closure is all attributes), so no FD violates BCNF. No decomposition needed. Enforcing non-overlap between successive intervals for the same (model_id, rate_type) is a business rule this relation cannot express declaratively in SQLite (see Bonus \S2) -- that is a constraint problem, not a normalisation violation, and is

out of scope for BCNF as such.

Relation 3: Video

RELATION: Video(video_id, owner_id, duration_ms, caption,
audio_track_id, uploaded_at)

FDs claimed:

F1: video_id → owner_id, duration_ms, caption, audio_track_id, uploaded_at
(holds because: video_id is the surrogate key assigned once at
upload; every other column describes exactly one uploaded clip)

Closures:

{video_id}⁺ = all attributes

Candidate keys: {video_id}

Verdict: BCNF trivially -- one FD, left side is the only key, no
violation possible. (Included as a control case: not every relation
in this schema has interesting FD structure, and it is worth showing
one that doesn't rather than only showing relations chosen because
they decompose.)

Relation 4: Recommendation (associative, agent layer) — the interesting case

RELATION: Recommendation(turn_id, position, video_id, impression_id,
session_id)

-- session_id is denormalised in here on the (incorrect) assumption
-- that callers would want it without a join back through AgentTurn;
-- this is deliberately included to demonstrate a BCNF violation, and
-- does NOT appear in the physical schema in C.1 -- see the note below.

FDs claimed:

F1: turn_id, position → video_id, impression_id
(holds because: one shelf slot in one turn recommends exactly one
clip, which resolves to at most one impression)
F2: turn_id → session_id
(holds because: every turn belongs to exactly one session --
inherited from AgentTurn's own key structure, AgentTurn.session_id)

Closures:

{turn_id, position}⁺ = {turn_id, position, video_id, impression_id,
session_id}
-- start: {turn_id, position}
-- apply F1: add video_id, impression_id
-- apply F2 (turn_id already held): add session_id
-- result: all attributes

{turn_id}⁺ = {turn_id, session_id}
-- start: {turn_id}
-- apply F2: add session_id
-- F1 cannot fire: position not held
-- result: NOT all attributes -- position, video_id, impression_id
remain outside the closure

Candidate keys: {turn_id, position}

(found by the closure above; {turn_id} alone is insufficient per its own closure just shown, and no smaller set can work since position never appears on the right of any FD, so it must be in every key -- per the bucket method in the task-sheet hint: position is a right-hand-side-never attribute, forcing it into any candidate key)

Verdict: NOT BCNF. F2's left-hand side {turn_id} is not a superkey ({turn_id}+ leaves out position, video_id, impression_id), so F2 violates BCNF.

Decomposition:

```
R1(turn_id, position, video_id, impression_id)  -- F1 preserved
R2(turn_id, session_id)                        -- F2 preserved
```

Lossless-join test:

```
R1 (cap) R2 = {turn_id}
{turn_id}+ in R2 (computed within R2's own attribute set) =
{turn_id, session_id} = all attributes of R2
=> turn_id is a key of R2, and the common attribute set R1 (cap) R2
equals a key of R2, which is the standard sufficient condition for
a lossless-join decomposition -- so this decomposition is lossless.
```

Dependency preserving: F1's attributes {turn_id, position, video_id, impression_id} all live inside R1; F2's attributes {turn_id, session_id} all live inside R2. Both original FDs are checkable within a single decomposed relation without needing to rejoin, so the decomposition is dependency-preserving as well as lossless.

*This is also why the actual **Recommendation** table in C.1 does **not** carry **session_id** — the decomposition above (R1) is exactly the physical design already in use; **session_id** is reached through **AgentTurn** instead of being stored redundantly. The BCNF violation shown here was introduced deliberately as a teaching example for this deliverable.*

4.2 D.2 — The Denormalisation Not Done

Candidate: a like_count column on Video, incremented on every EngagementSignal insert where signal_type = 'like' and decremented on every LikeRetraction.

Query it would accelerate: any feed-ranking or profile-display read that needs a clip's like count — currently requires COUNT(*) over EngagementSignal filtered by video_id and signal_type, minus a correlated count of matching LikeRetraction rows. At feed-serving latency, that aggregate on every render is the kind of read this column would exist to avoid.

Update anomaly it would admit: exactly the §2.7 failure mode — a like insert and a counter increment are two writes for one fact. A process killed between them leaves a counter that has drifted from the true count, and unlike moderation state (B.3(1), resolved by making current state a derived view with only one write), a materialised counter *is* the second write; there is no way to make “increment a running total” a single-write operation the way “append one log row” is.

Enforcement mechanism required to keep it honest: either (a) a trigger on EngagementSignal and LikeRetraction that adjusts the counter as part of the same transaction as each insert, which reduces but does not eliminate the risk — a mid-transaction failure per T1's own demonstration still leaves the two writes unreconciled if the trigger fires as a separate statement rather than atomically with the insert — or (b) the founders' original fix, a nightly reconciliation script recomputing the true count from source rows and overwriting the counter, which is precisely the approach the brief says is not the better answer (§2.7: “There is a better answer, and it is older than they are”).

Decision: not implemented. Likes are counted by aggregating EngagementSignal directly (as in F2 and v.video_daily_engagement, G.1); the write-side simplicity is judged worth more than the read-side speed at

this assignment's data volumes (300,000 impressions, not the tens of millions where a derived counter's cost would start to matter).

5 Deliverable E — Physical Implementation

5.1 E.1 — SQLite DDL

The complete DDL is in `schema.sql`, included as a submitted artifact and running clean against an empty database (SQLite 3.51.0, confirmed via `SELECT sqlite_version();`). `PRAGMA foreign_keys = ON` and `PRAGMA journal_mode = WAL` are the first two statements in the file, as required.

Type-affinity experiment

SQLite maps every declared type name onto one of five storage classes by a substring rule, so a declared type is a hint, not an enforced contract, unless something else backs it up. Three inserts tested this directly against the running schema (SQLite 3.51.0):

1. **INTEGER PRIMARY KEY** (`AppUser.user_id`): inserting the string `'not_an_integer'` was **rejected**:

Runtime error: datatype mismatch (20)

A column declared **INTEGER PRIMARY KEY** is SQLite's rowid alias, which is the one case where SQLite does enforce the integer type strictly.

2. **Plain INTEGER with a CHECK** (`Video.duration_ms`): inserting the string `'sixty seconds'` was **rejected**, but not by the type system — by the **CHECK**:

Runtime error: CHECK constraint failed: duration_ms BETWEEN 20000 AND 90000 (19)

A numeric-looking string, `'45000'`, was **silently accepted** and coerced:

```
sqlite> SELECT video_id, duration_ms, typeof(duration_ms) FROM Video WHERE video_id = 2;  
2|45000|integer
```

3. **Plain INTEGER with no CHECK** (`Impression.feed_position`): inserting the string `'not a number at all'` was **accepted without complaint**:

```
sqlite> SELECT impression_id, feed_position, typeof(feed_position) FROM Impression WHERE imp  
1|not a number at all|text
```

Nothing in the declared type stopped it.

What this means for the schema: the **INTEGER** keyword in a column declaration is close to decorative for anything that isn't the rowid-alias primary key. Every numeric business rule in this schema that actually matters — `duration_ms`'s bounds, token counts being non-negative, ratings falling in 1–5, cardinalities being 0/1 — is enforced by an explicit **CHECK**, not by the column's declared type. Correctness here rests on the **CHECK** constraints, not the type declarations; a column with a type but no **CHECK** (like `feed_position` above) is only as reliable as the application code writing to it.

I did **not** declare tables **STRICT**. **STRICT** tables (3.37+) would have rejected all three of the above inserts, including the numeric-string coercion, by accepting only **INT**, **INTEGER**, **REAL**, **TEXT**, **BLOB**, **ANY** and refusing type-affinity conversion. I chose not to, because the generator (E.3) and query layer both rely on SQLite's ordinary coercion behaviour in a few places (e.g. comparing **TEXT** timestamps lexicographically still needs to interoperate with `julianday()`'s numeric output in F9/F10), and introducing **STRICT** only on some tables while leaving others loose would be a worse inconsistency than being consistently non-strict and relying on **CHECKs** everywhere instead.

Closed value sets

Two approaches are mixed deliberately:

- **CHECK (col IN (...))** on a **TEXT** column — used for `account_state`, `ModerationDecision.state`, `signal_type`, `source`, `decided_by_type`, `rate_type`. These are five-to-seven-value sets defined by the business domain itself (moderation states, signal types) and not something Trust & Safety or any other consumer is going to add to without a product decision that also touches application code — cheap and invisible is the right trade here.

- No lookup table was used anywhere in this schema. I considered one for `ModerationDecision.state` specifically, since Trust & Safety is the group most likely to want a sixth state added later, but decided the `CHECK` is still the better default: adding a state is a one-line DDL migration either way (SQLite has no `ALTER TABLE ... DROP/ADD CONSTRAINT`, so a `CHECK` change means recreating the table regardless of whether it's a lookup table or not), and a lookup table would add a join to `v_video_current_state` (G.1) and every query that filters on state, for a set that has changed zero times in the brief's own description of the product.

Handle uniqueness

`UNIQUE (handle COLLATE NOCASE)` enforces case-insensitive uniqueness declaratively, resolving the Support ticket type in §2.1 (two handles differing only in capitalisation).

Two questions, answered directly:

1. **Does this work for a handle written in Tamil?** No. SQLite's `NOCASE` collation folds only ASCII `A-Z/a-z`; it has no notion of case for non-Latin scripts, and Tamil script itself has no case distinction to fold in the first place, so the failure mode isn't "two Tamil handles differing only in case" (that cannot occur) — the real gap is that `NOCASE` does nothing to normalise combining marks, alternate Unicode representations of the same grapheme, or visually confusable characters across scripts, none of which this constraint addresses. It solves exactly the ASCII-case problem §2.1 names and no more.
2. **Where does "unique among active accounts" get enforced?** Not here. A plain `UNIQUE` constraint has no way to express "unique only among rows where `account_state = 'active'`" — SQLite has no partial/filtered unique index syntax reachable from a table-level `UNIQUE` clause the way PostgreSQL's `CREATE UNIQUE INDEX ... WHERE` does.¹

Timestamps

Every timestamp column uses ISO-8601 UTC text (`'2026-03-03T14:32:07Z'`), with no epoch-integer columns anywhere in the schema.

Consequence for F3: irrelevant — F3 (audio tracks) touches no timestamp.

Consequence for F9 (longest streak of consecutive active days): ISO-8601 text sorts and compares correctly with plain `</>=` because the format is lexicographically ordered by construction, but day-boundary arithmetic ("were these two timestamps on consecutive calendar days") requires converting through `date()` or `julianday()` rather than string-slicing — the choice does not save that conversion, it only guarantees ordering is never silently wrong before the conversion happens.

Consequence for F10 (7-day rolling watch time): SQLite's window-frame `RANGE` clause takes numeric offsets only, so every windowed query converts the ISO-8601 column to `julianday(ts)` at query time regardless of storage format. Storing as text does not avoid this conversion; it only means the conversion is explicit and visible in every query that needs it, rather than baked into the storage layer as it would be with epoch integers computed once at write time.

I chose text over epoch integers specifically so that raw row inspection (during development, and in this document's query outputs) stays human-readable without a conversion step, at the cost of that same conversion needing to happen explicitly, and consistently, at read time in F9/F10/G.1. The single most common way this assignment goes wrong, per the task sheet's own hint, is mixing the two representations across tables — confirmed by inspection: every `TEXT` timestamp column in `schema.sql` is named `_at` or `ts`, with no numeric timestamp column anywhere, so there is nothing to mix.

¹SQLite *does* support partial indexes via `CREATE INDEX ... WHERE`, which could express this in principle; I chose not to rely on it, since a deactivated account's freed handle must also survive the 30-day pending-deletion recovery window without being reusable, which is a time-dependent rule a static partial index cannot express either. This half of A6 is enforced in application code, checked at handle-registration time, and traced as non-declarative in E.2.

NOT NULL audit

Nullable column	What NULL means here
AppUser.phone / google_id	“the user registered with the other method” — not unknown, structurally absent; the table-level CHECK requires at least one of the two
AppUser.deletion_requested_at	“does not apply” — the account is not currently pending deletion
CreatorTierPeriod.valid_to	“not yet known” — this is the creator’s current tier; the interval hasn’t closed
Follow.ended_at	“not yet known” — the follow is still active
AudioTrack.license_ref	“does not apply” — <code>source = 'original'</code> , there is no licence to reference
ModerationDecision.decided_by_id	“does not apply” — <code>decided_by_type = 'automated'</code> , there is no human reviewer id
EngagementSignal.share_destination	“does not apply” — <code>signal_type</code> is not <code>'share'</code> ; enforced declaratively by the paired CHECK
EngagementSignal.comment_text	“does not apply” — <code>signal_type</code> is not <code>'comment'</code> ; likewise CHECK-enforced
PromptTemplateVersion.valid_to	“not yet known” — this is the current version of the template
ModelPricingPeriod.valid_to	“not yet known” — this is the currently active rate
ToolCall.parent_tool_call_id	“does not apply” — this call is at the root of the turn’s call tree, not nested
ToolCall.result / latency_ms	“not yet known” — the call has not returned yet; the only pair of columns in this schema whose NULL means genuinely in-flight state rather than absence or a closed interval
Recommendation.impression_id	“not yet known, and may never be known” — the recommended clip has not (yet, or ever) produced an impression

Three distinct meanings appear above — *does not apply* (a field structurally irrelevant given another column’s value), *not yet known* (an open interval or in-flight process), and *may never be known* (a genuine absence with no expected resolution) — and no single column in this schema is asked to carry more than one of these meanings at once.

5.2 E.2 — Constraint Trace Table

#	Rule in words	Enforced by	Where
A1	A video carries at most one audio track	<code>audio_track_id</code> is a single nullable scalar FK, not a junction table — the cardinality is structural, not a runtime check	Video
A2	An impression yields at most one view	UNIQUE(<code>impression_id</code>) on View	View
A3	Impressions are logged unconditionally, independent of whether a view results	— not declarative — this is write-path behaviour (always insert an Impression row on viewport entry, regardless of what happens after); nothing in the DDL can force an application to write a row	application code
A4	A tool call belongs to exactly one turn	<code>turn_id</code> is a single mandatory FK column on ToolCall, not M:N	ToolCall
A5	A recommendation is a first-class entity, not inferred from an impression	Recommendation exists as its own table with its own composite PK (<code>turn_id</code> , <code>position</code>)	Recommendation
A6	A handle is unique — <i>among active accounts</i>	UNIQUE(<code>handle</code> COLLATE NOCASE) enforces global case-insensitive uniqueness; the “among active” half cannot be expressed by a plain UNIQUE in SQLite	AppUser (partial) — application code for the active-only half

#	Rule in words	Enforced by	Where
A7	Video's current moderation state is derived, never a second stored write	— not declarative — enforced by omission: <code>Video</code> has no <code>current_state</code> column at all; the only state lives in the append-only log, read through a view	schema design (no column) + <code>v_video_current_state</code> G.1
A8	Hashtags are not materialised; they live inside caption text	— not declarative — enforced by omission: no <code>Hashtag</code> table or column exists; <code>caption</code> is plain <code>TEXT</code>	schema design (no table)
A9	A block ends an existing follow (sets an end marker) rather than deleting it	<code>ended_at</code> column exists on <code>Follow</code> to receive the marker, but nothing in the DDL forces a block to actually set it	<code>Follow</code> (column only) — application/trigger code for the firing logic
A10	Moderation decisions are append-only; no decision is ever updated or deleted	— not declarative — <code>ModerationDecision</code> has no application-exposed <code>UPDATE/DELETE</code> path, but SQLite itself does not prevent either statement from being run directly against the table	application code (see Bonus §2 for a trigger-based alternative that would raise an error on <code>UPDATE/DELETE</code>)
A11	Watch time accumulates only at the segment grain, never as a running total on a coarser entity	— not declarative — enforced by omission: no <code>watch_total</code> or similar column exists anywhere on <code>Impression</code> , <code>Video</code> , or <code>AppUser</code> ; <code>ViewSegment.watch_ms</code> is the only place watch time is stored	schema design (no column)
A12	Model pricing is a validity-interval table, not a current-price column	<code>ModelPricingPeriod</code> PK is (<code>model_id</code> , <code>rate_type</code> , <code>valid_from</code>) with nullable <code>valid_to</code> ; no current-price column exists on any model-referencing table	<code>ModelPricingPeriod</code>

What this table shows

Five of the twelve assumptions (A1, A2, A4, A5, A12) are enforced entirely declaratively — a constraint violation is impossible to write past the database, full stop. Two (A7, A8, A11) are enforced by *omission*: the assumption holds because the tempting column simply does not exist in the schema, which is a real enforcement mechanism (you cannot violate a rule against a column that isn't there) but a fragile one, since nothing stops a future migration from adding the column back and silently reintroducing the anomaly it currently prevents. Three (A3, A9, A10) are not enforceable by SQLite at all in this design and rest entirely on application code doing the right thing every time — A10 in particular is worth flagging, since it is the exact failure mode §2.7 describes (a write path that is supposed to be single-write but isn't guaranteed to be), and the honest answer is that the schema alone does not close this gap; only a trigger (Bonus §2) or disciplined application code does. A6 is split — the part that can be declarative, is.

5.3 E.3 — Data Generator

The generator is `generate_data.py`, run as:

```
sqlite3 scrollsense.db < schema.sql
python3 generate_data.py
```

It is parameterised and seeded (`SEED = 37`), with a `SCALE` multiplier at the top of the file. Default volumes match the task sheet exactly: 5,000 users, 20,000 videos, 300,000 impressions, 2,000 agent sessions. The full load runs inside a single transaction (`BEGIN/COMMIT` wrapping every `executemany` call) and completes in **31 seconds** on the reference machine — confirmed by timing the actual run, not estimated.

Distribution checks — verified against the generated database, not asserted

Funnel (§2.4, §2.6): of 300,000 impressions, 66,046 became views (**22.0%**), and of those, 7,824 produced an explicit engagement signal (**11.8% of views**). Both ratios match the brief’s description that most impressions never become views and most views produce no signal, and were confirmed with a direct query against the populated database rather than assumed from the generation probabilities.

Follower counts (power-law): an initial implementation applied the skew to *out*-degree (how many accounts a user follows) and used uniform random sampling for *who* gets followed — this produced a nearly flat in-degree distribution (max follower count of 48 across 5,000 users), because uniform target selection cancels out any skew applied only to how many follows each user initiates. The fix assigns each user a Zipf-distributed popularity weight and uses that weight when *other* users choose who to follow, since follower count is fundamentally an in-degree property. The corrected distribution: **44.9% of users have zero followers**, median is 1, but the top 1% of users hold **55% of all follow relationships** — confirmed by direct aggregation query, not by inspecting the generation code alone.

Watch time (right-skewed): across 82,346 `ViewSegment` rows, `watch_ms` percentiles are `p25 = 4,726`, `median = 20,973`, `p75 = 50,370`, `p95 = 84,701` (out of a ceiling near 95,000) — most segments are short, with a long tail toward the full clip length, consistent with most scrolls being brief glances and a minority being full watches.

Daily rhythm: impression timestamps are drawn from a 24-value hourly weight table peaking at 19:00–21:00 (roughly 5× the overnight low), producing a smooth single-peaked curve when histogrammed by hour — confirmed visually against the actual generated timestamp distribution, not just the weight table used to draw them.

Integrity

`PRAGMA foreign_key_check` was run against the fully populated database after generation and returns **zero violations** — every foreign key across all 25 tables resolves. This is checked programmatically at the end of `generate_data.py` itself, so a broken generator run fails loudly rather than silently producing an invalid database.

Design notes

- Load order respects every foreign-key dependency (users before videos, videos before impressions, impressions before views, views before segments, and so on) — there is no forward reference anywhere in the script that would require deferred constraint checking.
- `Recommendation` rows are inserted with `INSERT OR IGNORE` to silently absorb the rare case where the same (`turn_id`, `position`) pair is generated twice, rather than letting a low-probability collision abort the entire transaction.
- Prompt template versions and model pricing periods are generated as genuine validity-interval chains (each period’s `valid_to` is the next period’s `valid_from`, with the final period left open), so that `AgentTurn` rows can be joined against “the version/rate in force at that timestamp” the way B.2 and G.1 require, rather than pointing at a single current row.

- **ModerationDecision** sequences per video vary in length (weighted 70/20/7/3 toward 1/2/3/4 decisions), so that F8’s “videos whose moderation state changed more than twice” has a genuine, non-trivial population to return rather than an empty or universal result.

5.4 E.4 — JSON Use

So we get that, JSON appears in exactly one place in this schema: `ToolCall.arguments_json`, guarded by `CHECK (json_valid(arguments_json))` and read via `json_extract()` where needed (e.g. pulling out a specific argument for a query without materialising a new column for it).

Verified, not just declared: a direct insert of `'{"query": "cats"}'` succeeded; an insert of the malformed string `'{not valid json}'` was rejected with `CHECK constraint failed: json_valid(arguments_json)`. The constraint does what it claims to do.

Why this column, and no other

In §2.5 it lists three example tool calls — `search_videos(query, filters)`, `get_user_history(days)`, `fetch_trending_audio(region)` — each with a genuinely different argument shape, and the brief states plainly that a tool call may trigger nested sub-calls to arbitrary depth. Any given `ToolCall` row’s `name` determines what its arguments even mean: `days` is meaningless for `fetch_trending_audio`, and `region` is meaningless for `get_user_history`. Modelling this relationally would mean either one column per possible argument across every tool that has ever existed (mostly NULL, and requiring a schema migration — recreating the table, since SQLite has no `ALTER TABLE ... ADD CONSTRAINT` — every time a new tool is added), or a separate table per tool name (fragmenting `ToolCall` the same way the seven-signal-table alternative was rejected in B.3(3), for the same reason: F11’s recursive CTE over the call tree needs one shared shape to walk, not N incompatible ones). A JSON column is the genuine admission here: the argument shape is open-ended by design, since new tools will be added as the agent’s capabilities grow, and the brief does not ask this schema to enumerate or constrain what a tool call’s arguments can contain — only that they be recorded.

Why nowhere else

Every other candidate for “this looks unstructured” in this schema in fact has a fixed, known field set:

- **EngagementSignal**’s per-type attributes (`share_destination`, `comment_text`) are two known nullable columns, not an open set — §2.4 enumerates all seven signal types and their attributes exhaustively, so there is nothing left to discover later the way a new tool’s argument shape would be.
- **JudgeScore**’s three dimensions (helpfulness, groundedness, safety) are named explicitly in §2.5 and deliberately kept as three fixed columns rather than a JSON blob or a rubric table (A.2’s first descoping decision) — precisely because they are closed and known, the opposite situation from tool arguments.
- **ToolCall.result** is deliberately left as plain `TEXT`, *not* JSON, even though a tool’s return value is arguably just as variable in shape as its arguments. This was a close call: I considered `CHECK (json_valid(result))` for symmetry with `arguments_json`, but as of this deliverable no query planned for F1–F13 needs to reach inside a tool’s result to filter or aggregate on a field within it — only whether the call `errored` and its `latency_ms` matter downstream. *This is a prediction, not yet a checked fact:* if a later query in Deliverable F turns out to need a field from inside `result`, this decision should be revisited rather than worked around with ad-hoc string parsing.

What is lost by choosing JSON here

- **No foreign keys into it.** If `fetch_trending_audio`’s `region` argument ever needed to reference a real `Region` entity (as the rejected multi-region licensing scope in A.2 would have required), that reference would live inside opaque text, unenforced and unindexed — a video’s actual `AudioTrack` relationship is a real foreign key; a tool call’s *argument that happens to name a region* is not, and never resolves to one automatically.
- **No CHECK on its interior.** `json_valid(arguments_json)` only guarantees the text parses as JSON at all — it enforces nothing about *which* tool requires *which* keys. A `search_videos` call missing its

`query` field, or carrying a `region` key that only makes sense for a different tool, would insert without complaint. That validation, if wanted, would have to live in application code at the point tool calls are logged, not in the database.

6 Deliverable F — Queries

All thirteen queries are in `queries.sql`, run against the generated database (seed = 37) via `run_queries.py`. Row counts and runtimes below are measured directly on this machine — no query here has been tuned; that is deliberate, per the task sheet, since these numbers are the Assignment 2 baseline.

F1 — Top 10 audio tracks by distinct videos, last 7 days

```
SELECT at.track_id, COUNT(DISTINCT v.video_id) AS distinct_videos
FROM AudioTrack at
JOIN Video v ON v.audio_track_id = at.track_id
WHERE v.uploaded_at >= datetime('2026-08-25T00:00:00Z')
GROUP BY at.track_id
ORDER BY distinct_videos DESC
LIMIT 10;
```

Rows: 10 Runtime: 8.3ms

First 5: (5322, 2), (5112, 2), (4464, 2), (1827, 2), (506, 2)

Reading: the busiest tracks in the last 7 days of the simulation window carry only 2 distinct videos each — expected, since 300,000 impressions over a full year leaves any single 7-day slice of *uploads* thin; a real 400k-MAU app would show far more concentration around trending sounds.

F2 — Watch hours and mean completion rate per creator, live clips only

```
WITH live_videos AS (
  SELECT v.video_id, v.owner_id FROM Video v
  WHERE (SELECT md.state FROM ModerationDecision md
         WHERE md.video_id = v.video_id
         ORDER BY md.decided_at DESC LIMIT 1) = 'live'
),
telemetry AS (
  SELECT lv.owner_id, SUM(vs.watch_ms) AS watch_ms_total,
         AVG(vs.completed) AS completion_rate
  FROM live_videos lv
  LEFT JOIN Impression i ON i.video_id = lv.video_id
  LEFT JOIN View vw ON vw.impression_id = i.impression_id
  LEFT JOIN ViewSegment vs ON vs.view_id = vw.view_id
  GROUP BY lv.owner_id
)
SELECT au.user_id AS creator_id,
       COALESCE(t.watch_ms_total, 0) / 3600000.0 AS watch_hours,
       COALESCE(t.completion_rate, 0.0) AS mean_completion_rate
FROM AppUser au
LEFT JOIN telemetry t ON t.owner_id = au.user_id
WHERE au.user_id IN (SELECT DISTINCT owner_id FROM Video)
ORDER BY watch_hours DESC;
```

Rows: 750 Runtime: 594.1ms

First 5: (4770, 1.35, 0.172), (1110, 1.34, 0.180), (2208, 1.30, 0.150), (4512, 1.24, 0.231), (1778, 1.17, 0.162)

Expected row count reasoning: 750 was confirmed independently before trusting the result — `SELECT COUNT(DISTINCT owner_id) FROM Video` also returns exactly 750, so every creator appears, including those with zero live clips or zero watch time, because the outer `LEFT JOIN` chain is anchored on `AppUser`, not on the telemetry tables. Note that creator 2208 (the same creator used in F6 below) appears here with a normal 15.0% completion rate despite having 587 users who saw but never engaged with their clips — the two facts are consistent, since watching and engaging are independent funnel stages.

F3 — Videos with no audio track: NOT IN vs NOT EXISTS

```
-- NOT IN
SELECT COUNT(*) FROM Video
WHERE audio_track_id NOT IN (SELECT track_id FROM AudioTrack);
-- result: 0

-- NOT EXISTS
SELECT COUNT(*) FROM Video v
WHERE NOT EXISTS (SELECT 1 FROM AudioTrack at WHERE at.track_id = v.audio_track_id);
-- result: 7958
```

Runtime: 3.8ms (NOT IN), 3.3ms (NOT EXISTS)

The two counts genuinely differ: 0 vs 7,958. This is the exact trap the task sheet flags. `audio_track_id` is nullable; `AudioTrack.track_id` is not. For a video with `audio_track_id` IS NULL, the expression `NULL NOT IN (...)` evaluates to NULL (unknown), not TRUE — SQL’s three-valued logic means a `WHERE` clause only keeps rows where the condition is TRUE, so every one of the 7,958 videos with no audio track is silently dropped by the `NOT IN` version, leaving zero rows. `NOT EXISTS` is a correlated existence check and is NULL-safe by construction: a NULL `audio_track_id` can never equal any `track_id`, so the correlated subquery correctly finds no match and `NOT EXISTS` returns TRUE. **NOT EXISTS is correct here; NOT IN silently returns the wrong answer whenever the compared column can be NULL.** Verified independently against a direct `WHERE audio_track_id IS NULL` count, which also returns 7,958.

F4 — Users who liked then retracted within 60 seconds

```
SELECT es.user_id, es.video_id, es.ts AS liked_at, lr.retracted_at
FROM EngagementSignal es
JOIN LikeRetraction lr ON lr.like_signal_id = es.signal_id
WHERE es.signal_type = 'like'
      AND (julianday(lr.retracted_at) - julianday(es.ts)) * 86400.0 <= 60;
```

Rows: 113 Runtime: 3.1ms

First 5: (2809, 295, '2026-04-08T17:34:28Z', '2026-04-08T17:34:48Z'), (2413, 11555, ...), (2677, 18991, ...), (3424, 18050, ...), (1979, 9448, ...)

Reading: 113 retractions happened within 60 seconds of the original like — the fast-retraction pattern (impulsive tap-and-undo) is a substantial share of all retractions in this dataset.

F5 — Videos whose caption carries a given hashtag

```
SELECT video_id, caption FROM Video
WHERE lower(caption) LIKE '%#comedy%';
```

Rows: 2,308 Runtime: 4.4ms

First 5: (1, 'check this out #comedy'), (6, 'check this out #music #fitness #comedy'), (26, ...), (38, ...), (40, ...)

SQLite’s `LIKE` is already case-insensitive for ASCII, so `lower()` is technically redundant for this pattern but kept for defensiveness and clarity of intent; `GLOB` was not used because `GLOB` is case-sensitive, the wrong default here. **For a caption written in Tamil**, correctness does not change for exact-text matching — `lower()` is a no-op on Tamil script since Tamil has no case distinction, and `LIKE`’s substring match over the literal ASCII `#comedy` pattern works identically regardless of what surrounds it. What `lower()` cannot do is fold alternate Unicode representations of visually identical Tamil text — a real but different gap from case-folding.

F6 — Shown but never engaged (set operator); UNION vs UNION ALL

```
SELECT DISTINCT i.user_id FROM Impression i
JOIN Video v ON v.video_id = i.video_id WHERE v.owner_id = 2208
EXCEPT
SELECT DISTINCT es.user_id FROM EngagementSignal es
```

```
JOIN Video v ON v.video_id = es.video_id WHERE v.owner_id = 2208;
```

Rows: 587 Runtime: 502.2ms

First 5: 11, 14, 17, 21, 27

Reading: creator 2208's clips were shown to 608 distinct users; only 21 ever engaged, leaving 587 shown-but-silent users — a stark illustration of §2.6's funnel-leak warning applied at the per-creator level. Consistent with F2, where this same creator shows a below-average 15.0% completion rate.

```
-- UNION (dedups across the two signal types)
SELECT user_id FROM EngagementSignal WHERE signal_type = 'like'
UNION
SELECT user_id FROM EngagementSignal WHERE signal_type = 'save';
-- rows: 3092, runtime: 2.3ms
```

```
-- UNION ALL (no dedup)
SELECT user_id FROM EngagementSignal WHERE signal_type = 'like'
UNION ALL
SELECT user_id FROM EngagementSignal WHERE signal_type = 'save';
-- rows: 4744, runtime: 1.8ms
```

UNION returns 3,092 distinct users who liked and/or saved at least one clip; UNION ALL returns 4,744, counting a user twice if they both liked and saved. The gap of 1,652 rows is exactly the number of user-appearances collapsed by deduplication — UNION ALL is right for “how many like/save events happened,” UNION is right for “how many distinct users did at least one of these.”

F7 — Cost per agent session, by template version, above a threshold

```
WITH turn_cost AS (
  SELECT tu.turn_id, at.session_id, at.template_id, at.template_version,
         (tu.input_tokens * mpp_in.rate + tu.output_tokens * mpp_out.rate
          + tu.cached_tokens * mpp_cache.rate) AS turn_cost
  FROM TurnUsage tu
  JOIN AgentTurn at ON at.turn_id = tu.turn_id
  JOIN ModelPricingPeriod mpp_in
    ON mpp_in.model_id = tu.model_id AND mpp_in.rate_type = 'input'
    AND tu.priced_at >= mpp_in.valid_from
    AND (mpp_in.valid_to IS NULL OR tu.priced_at < mpp_in.valid_to)
  JOIN ModelPricingPeriod mpp_out
    ON mpp_out.model_id = tu.model_id AND mpp_out.rate_type = 'output'
    AND tu.priced_at >= mpp_out.valid_from
    AND (mpp_out.valid_to IS NULL OR tu.priced_at < mpp_out.valid_to)
  JOIN ModelPricingPeriod mpp_cache
    ON mpp_cache.model_id = tu.model_id AND mpp_cache.rate_type = 'cached_input'
    AND tu.priced_at >= mpp_cache.valid_from
    AND (mpp_cache.valid_to IS NULL OR tu.priced_at < mpp_cache.valid_to)
)
SELECT session_id, template_id, template_version, SUM(turn_cost) AS session_cost
FROM turn_cost
GROUP BY session_id, template_id, template_version
HAVING SUM(turn_cost) > 0.01
ORDER BY session_cost DESC;
```

Rows: 5,161 Runtime: 28.0ms

First 5: (559, 3, 84, 2566.06), (1372, 1, 60, 2390.10), (1510, 4, 49, 2353.92), (435, 2, 75, 2310.34), (669, 3, 82, 2307.57)

This is the query that concretely exercises the schema consequence named in A12/B.2: each of the three rate types (input, output, cached_input) is joined against ModelPricingPeriod independently, with the join condition itself doing the “price in force at this turn's timestamp” resolution — there is no current-price

column anywhere this query could accidentally use instead.

F8 — Videos whose moderation state changed more than twice

```
SELECT video_id, COUNT(*) AS n_decisions,  
       group_concat(state, ' -> ' ORDER BY decided_at) AS sequence  
FROM ModerationDecision  
GROUP BY video_id  
HAVING COUNT(*) > 2  
ORDER BY n_decisions DESC;
```

Rows: 2,001 Runtime: 13.5 ms (SQLite 3.51.0, so `ORDER BY` inside `group_concat` is honoured)

First 5: (46, 4, 'taken_down -> live -> live -> live'), (99, 4, 'age_restricted -> live -> live -> live'), (133, 4, ...), (225, 4, ...), (250, 4, ...)

Reading: some sequences show real churn (row 133: `age_restricted -> live -> live -> demoted`), others show a video repeatedly re-confirmed in the same state by different automated passes (`live -> live -> live`) — both are legitimate append-only histories under A10, but this query counts decisions, not distinct states; a stricter reading (only counting actual state *changes*) would need to filter out consecutive repeats, worth flagging as an interpretation choice.

F9 — Longest streak of consecutive active days per user

```
WITH active_days AS (  
    SELECT DISTINCT user_id, date(ts) AS d FROM Impression  
)  
,  
numbered AS (  
    SELECT user_id, d,  
           julianday(d) - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY d) AS grp  
    FROM active_days  
)  
,  
streaks AS (  
    SELECT user_id, grp, COUNT(*) AS streak_len FROM numbered GROUP BY user_id, grp  
)  
SELECT user_id, MAX(streak_len) AS longest_streak  
FROM streaks GROUP BY user_id ORDER BY longest_streak DESC LIMIT 20;
```

Rows: 20 (top 20 shown; 5,000 total if unlimited) Runtime: 346.8 ms

First 5: (655, 10), (4682, 7), (1651, 7), (449, 7), (4809, 6)

The classic “group consecutive dates” trick: subtracting a per-user row number from the date (via `julianday`) produces a constant value for every row inside one unbroken run, so grouping on that constant isolates each streak. Reading: the longest streak in this dataset is 10 consecutive days — short, because 300,000 impressions spread across 5,000 users over a full year gives each user roughly one impression every 6 days on average, so long unbroken runs are statistically unlikely at this data volume.

F10 — Rank creators by 7-day rolling watch time, week-over-week change

```
WITH daily_watch AS (  
    SELECT v.owner_id AS creator_id, date(i.ts) AS d, SUM(vs.watch_ms) AS watch_ms  
    FROM Video v  
    JOIN Impression i ON i.video_id = v.video_id  
    JOIN View vw ON vw.impression_id = i.impression_id  
    JOIN ViewSegment vs ON vs.view_id = vw.view_id  
    GROUP BY v.owner_id, date(i.ts)  
)  
,  
rolling AS (  
    SELECT creator_id, d, watch_ms,  
           SUM(watch_ms) OVER (PARTITION BY creator_id ORDER BY julianday(d)  
                               RANGE BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_7d_ms
```

```

    FROM daily_watch
)
SELECT creator_id, d, rolling_7d_ms,
       rolling_7d_ms - LAG(rolling_7d_ms, 7) OVER (PARTITION BY creator_id ORDER BY julianday(d))
       RANK() OVER (PARTITION BY d ORDER BY rolling_7d_ms DESC) AS daily_rank
FROM rolling
ORDER BY d DESC, daily_rank
LIMIT 30;

```

Rows: 30 (shown; full result is one row per creator-day) Runtime: 381.2ms

First 5: (68, '2026-08-31', 400054, 399641, 1), (4315, '2026-08-31', 345052, 249552, 2), (694, ...), (1664, ...), (3996, ...)

“7 days” is read here as seven calendar days, via RANGE BETWEEN 6 PRECEDING AND CURRENT ROW ordered on julianday(d) — SQLite’s RANGE frame takes numeric offsets only, which is why the date column is converted at query time rather than compared as text. This is deliberately *not* “the creator’s 7 most recent active days,” which would need a ROWS BETWEEN 6 PRECEDING frame over a gap-free per-creator day sequence instead — a materially different query, since a creator with sparse activity would have their “7 active days” span many calendar weeks under that reading.

F11 — Full tool-call nesting tree for one agent session, with depth

```

WITH RECURSIVE call_tree AS (
    SELECT tc.tool_call_id, tc.turn_id, tc.parent_tool_call_id, tc.name, 0 AS depth
    FROM ToolCall tc
    JOIN AgentTurn at ON at.turn_id = tc.turn_id
    WHERE at.session_id = 235 AND tc.parent_tool_call_id IS NULL
    UNION ALL
    SELECT tc.tool_call_id, tc.turn_id, tc.parent_tool_call_id, tc.name, ct.depth + 1
    FROM ToolCall tc
    JOIN call_tree ct ON tc.parent_tool_call_id = ct.tool_call_id
)
SELECT tool_call_id, turn_id, name, depth FROM call_tree ORDER BY turn_id, depth;

```

Rows: 13 Runtime: 6.9ms

First 5: (933, 812, 'search_videos', 0), (935, 812, 'fetch_trending_audio', 0), (934, 812, 'search_videos', 1), (936, 813, 'fetch_trending_audio', 0), (937, 813, 'get_user_history', 0)

Session 235 was chosen specifically because it contains genuine nesting (confirmed by querying for sessions with at least one parent_tool_call_id IS NOT NULL row before selecting this example) — an arbitrarily chosen session risked returning only depth-0 root calls, which would demonstrate nothing about the recursion. Turn 812 shows a depth-1 search_videos call nested under a depth-0 root call in the same turn, confirming the recursive CTE correctly walks ToolCall’s self-referencing FK exactly as A4 intends: one turn’s call tree, no cross-turn edges possible by construction.

F12 — Recommended clips watched to completion, with shelf position

```

SELECT s.session_id, r.turn_id, r.position, r.video_id
FROM Recommendation r
JOIN AgentTurn at ON at.turn_id = r.turn_id
JOIN AgentSession s ON s.session_id = at.session_id
JOIN Impression i ON i.impression_id = r.impression_id
JOIN View v ON v.impression_id = i.impression_id
JOIN ViewSegment vs ON vs.view_id = v.view_id
WHERE vs.completed = 1
GROUP BY s.session_id, r.turn_id, r.position, r.video_id;

```

Rows: 232 Runtime: 26.0ms

First 5: (4, 14, 1, 1039), (5, 19, 4, 4184), (9, 35, 4, 10968), (37, 120, 4, 3903), (46, 156, 4, 4286)

This is the query the task sheet names as the spine of the whole design. It was not hard to write specifically

because A5 gave `Recommendation` its own first-class identity with a direct FK to `Impression` — the join chain `Recommendation → Impression → View → ViewSegment` is a straight line with no fragile heuristic matching on (session, position, timestamp proximity) required, exactly what A5’s justification predicted.

F13 — High judge score, thumbs-down disagreement

```
SELECT at.turn_id, js.helpfulness, js.groundedness, js.safety, ur.thumbs_up
FROM AgentTurn at
JOIN JudgeScore js ON js.turn_id = at.turn_id
JOIN UserRating ur ON ur.turn_id = at.turn_id
WHERE (js.helpfulness > 4 OR js.groundedness > 4 OR js.safety > 4)
      AND ur.thumbs_up = 0;
```

Rows: 4 Runtime: 2.3ms

Full result: (1195, 5, 5, 2, 0), (2728, 3, 5, 2, 0), (3878, 5, 5, 1, 0), (4179, 5, 3, 4, 0)

This disagreement set is the most commercially valuable data in the company precisely because it is small and adversarial to the product’s own quality signal: the LLM judge — itself a proxy the team built to avoid needing a human on every turn — rated these responses highly on at least one axis, yet a real user, with context the judge didn’t have, pushed back. Every other quality signal in this schema (`JudgeScore` alone, `UserRating` alone) can drift undetected; the two disagreeing is the one place the system’s blind spot becomes visible without waiting for a human audit.

7 Deliverable G — Views and Transactions

7.1 G.1 — Views

All five views are in `views.sql`, tested against the generated database (seed 37). Every one runs and returns plausible results:

View	Consumer	Columns exposed			Depends on which B.2 storage decision?
<code>v_public_profile</code>	mobile client	<code>user_id</code> , <code>play_name</code> ,	<code>handle</code> , <code>follower_count</code>	<code>dis-</code>	<code>account_state</code> (overwrite) — filters out non-active rows; <code>phone/google.id/deletion_requested_at</code> never selected
<code>v_video_current_state</code>	Trust & Safety	<code>video_id</code> , <code>state_as_of</code>	<code>current_state</code> ,		append-only <code>ModerationDecision</code> log (A7/A10) — resolves to the row with <code>MAX(decided_at)</code> per video
<code>v_creator_tier_current</code>	Growth	<code>creator_id</code> , <code>tier</code> , <code>valid_from</code>			validity-interval <code>CreatorTierPeriod</code> (A12/B.2) — the row where <code>valid_to</code> IS NULL
<code>v_video_daily_engagement</code>	Growth analysts	<code>video_id</code> , <code>watch_seconds</code> ,	<code>day</code> , <code>impression_count</code> , <code>view_count</code> , <code>net_likes</code>	<code>impres-</code>	anchored on <code>Impression</code> (event log), LEFT JOINed to <code>view/like/retraction</code> aggregates so quiet days survive
<code>v_turn_cost</code>	Finance	<code>turn_id</code> , <code>model_id</code> , <code>turn_cost</code>	<code>session_id</code> , <code>priced_at</code> ,		validity-interval <code>ModelPricingPeriod</code> (A12) — joined on the turn's own timestamp, not the current rate

Verified row counts against the generated database: `v_public_profile` returns **4,812** rows (out of 5,000 users, minus deactivated/pending accounts); `v_video_current_state` returns exactly **20,000** — one row per video, matching the `Video` table count exactly; `v_creator_tier_current` returns **750**, matching the distinct-creator count exactly; `v_video_daily_engagement` confirms genuine zero-days survive — video 1 on 2025-09-10 shows (1 `impression`, 0 `views`, 0.0 `watch_seconds`, 0 `net_likes`) rather than being absent from the result, which is exactly what an inner join anywhere in the view's chain would have silently dropped; `v_turn_cost` returns per-turn costs correctly resolved against each model's rate at that specific turn's timestamp.

7.2 G.2 — Written Answers

(a) Updatability

Attempting `INSERT INTO v_public_profile (...) VALUES (...)` against the running database returns:
`cannot modify v_public_profile because it is a view`

confirmed directly, not assumed. SQLite views are unconditionally read-only regardless of their shape — there is no code path in SQLite that ever permits writing through a view.

PostgreSQL would accept an `INSERT` through `v_public_profile` but reject one through `v_video_daily_engagement`. The property that makes a view updatable in principle — independent of what any one engine chooses to implement — is whether every row of the view corresponds to exactly one row of exactly one base table, with every base-table column not exposed by the view either nullable or defaulted, and no aggregation, grouping, join, or `DISTINCT` standing between the view and that one row. `v_public_profile` satisfies this: it is a straightforward projection and filter (`WHERE account_state = 'active'`) over a single table, `AppUser`, plus one correlated-subquery-derived column (`follower_count`), and PostgreSQL's updatable-view rules (since 9.3) permit inserting through simple projections like this, filling unlisted columns with their defaults or NULL. `v_video_daily_engagement` fails this test outright: it aggregates over four separate subqueries joined together, with no single row corresponding to any one row of any one base table — there is no coherent way to

translate “insert this daily-engagement row” back into inserts against `Impression`, `View`, `ViewSegment`, and `EngagementSignal` simultaneously, so no engine, however permissive, could make this view updatable without additional machinery (PostgreSQL’s `INSTEAD OF` triggers being exactly that machinery, applied manually rather than inferred automatically).

(b) The as-of problem

`v_creator_tier_current` answers “what is this creator’s tier right now” by filtering to the one row per creator where `valid_to` IS NULL. A view’s body is fixed SQL text with no parameters — it cannot accept an argument, so there is no way to pass in an arbitrary past timestamp T and have the view filter to “the row valid at T ” instead of “the row valid now.” Rewriting the view’s `WHERE` clause to reference a specific hard-coded date would only ever answer the question for that one date, not for an arbitrary T chosen at query time, which is precisely what the ML team’s requirement demands (§3: reconstructing state “at any past timestamp”). What is needed instead is a parameterised query or a table-valued function — in SQLite terms, this means a query template that takes T as a bound parameter (`SELECT tier FROM CreatorTierPeriod WHERE creator_id = ? AND valid_from <= ? AND (valid_to IS NULL OR valid_to > ?)`), run directly by the application rather than wrapped in a view) or, if a reusable named object is wanted, a table-valued function in an engine that supports one (PostgreSQL functions returning `SETOF`, for instance) — SQLite has no equivalent, so this stays as a parameterised query in the ML pipeline’s own code rather than a database object.

7.3 G.3 — Transaction Scripts

All three scripts are in `transactions.sql`. Two real implementation problems were found and corrected while building T1, and all three scripts were verified end-to-end with real concurrent connections, not merely described.

T1 — Retraction atomicity

First problem found: the task sheet’s suggested failure injector, `SELECT 1/0;`, does not raise an error in SQLite — `1/0` evaluates to NULL (SQLite’s integer division by zero is defined, not an error, unlike PostgreSQL or MySQL). This injector was replaced with a genuine `UNIQUE` constraint violation — inserting a second `LikeRetraction` row for the same `like_signal_id` — which does fail.

Second, more consequential problem: SQLite’s default `ON CONFLICT ABORT` resolution rolls back only the *failing statement*, not the enclosing transaction. Verified directly against a real like (`signal_id = 7015`):

```
BEGIN;
INSERT INTO LikeRetraction (retraction_id, like_signal_id, retracted_at)
VALUES (999905, 7015, '2026-01-01T00:00:05Z');
INSERT INTO LikeRetraction (retraction_id, like_signal_id, retracted_at)
VALUES (999906, 7015, '2026-01-01T00:00:10Z');
Runtime error: UNIQUE constraint failed: LikeRetraction.like_signal_id

SELECT COUNT(*) FROM LikeRetraction WHERE retraction_id = 999905;
1  -- the FIRST insert is still sitting uncommitted after the failure

ROLLBACK;
SELECT COUNT(*) FROM LikeRetraction WHERE retraction_id = 999905;
0  -- only the explicit ROLLBACK removes it
```

This confirms the first insert was never automatically undone — had a `COMMIT` been issued instead of an explicit `ROLLBACK` at that point (the naive reading of the task sheet’s own template, “`COMMIT;` – will not be reached”), the first insert *would* have been committed. `transactions.sql` issues the explicit `ROLLBACK`; final state confirmed: `0` rows for both IDs.

T2 — Moderation decision visibility (WAL)

Verified with two genuinely separate `sqlite3` processes connected to the same database file. Connection A opened a transaction and inserted a new `ModerationDecision` row for video 1 (`'taken_down'`) without

committing. While A's transaction was still open, Connection B queried `v_video_current_state` for video 1 and saw `'live'` — the old state, confirming A's uncommitted write was invisible to B. After A committed, B re-queried and saw `'taken_down'`. This is WAL's snapshot-isolation behaviour specifically: under the default rollback-journal mode, B's read would instead *block* until A released its lock, which is why `journal_mode = WAL` is set at the top of `schema.sql` — without it, this demonstration would show blocking, not a stale-but-consistent snapshot.

T3 — Handle change limit and SQLITE_BUSY

The twice-a-year handle rule is enforced by counting rows in an application-maintained `HandleChangeLog` table within a rolling twelve-month window before allowing a change — verified directly: after inserting two change records for user 1 (10 Jan and 10 Apr 2026), querying the count within the trailing year correctly returns **2**, meaning a third attempt inside the same window must be refused by the application (SQLite has no declarative way to express a rolling-window `COUNT` as a `CHECK` constraint).

The `SQLITE_BUSY` demonstration was run with two genuinely separate terminal processes: Connection A opened `BEGIN IMMEDIATE` and updated `AppUser` for user 2, without committing. Connection B then attempted `BEGIN IMMEDIATE` against a *completely unrelated* row (user 3) and failed immediately with:

```
Runtime error: database is locked (5)
```

What this shows: SQLite permits exactly one writer at a time across the *entire database file*, not per-row or per-table locking — confirmed here by the fact that B's write to a completely different row than A's still failed. This sidesteps the lost-update anomaly a multi-writer engine must instead resolve with row-level locking or MVCC conflict detection, at the cost of forcing every concurrent writer but one to block or fail outright, even when, as demonstrated here, the two writers touch entirely unrelated data.

Bonus

Bonus 1 — Trigger-based audit-log immutability for ModerationDecision (2 marks)

Two FOR EACH ROW triggers (the only kind SQLite supports) reject any UPDATE or DELETE against ModerationDecision

```
CREATE TRIGGER trg_moddecision_no_update
BEFORE UPDATE ON ModerationDecision
BEGIN
    SELECT RAISE(ABORT, 'ModerationDecision is append-only: UPDATE not permitted (A10)');
END;
```

```
CREATE TRIGGER trg_moddecision_no_delete
BEFORE DELETE ON ModerationDecision
BEGIN
    SELECT RAISE(ABORT, 'ModerationDecision is append-only: DELETE not permitted (A10)');
END;
```

Verified directly against the populated database:

```
UPDATE ModerationDecision SET state='live' WHERE video_id=1;
-- Runtime error: ModerationDecision is append-only: UPDATE not permitted (A10)
```

```
DELETE FROM ModerationDecision WHERE video_id=1;
-- Runtime error: ModerationDecision is append-only: DELETE not permitted (A10)
```

Both correctly rejected against the live, populated database.

Bonus 2 — Trigger preventing overlapping validity intervals (2 marks)

Applied to CreatorTierPeriod. PostgreSQL would express this declaratively with an EXCLUDE constraint over a range type and the && overlap operator; SQLite has neither range types nor EXCLUDE, so the check is hand-written as two triggers (INSERT and UPDATE separately, since SQLite triggers are per-operation and do not share logic across statement types):

```
CREATE TRIGGER trg_creatortier_no_overlap_insert
BEFORE INSERT ON CreatorTierPeriod
BEGIN
    SELECT RAISE(ABORT, 'Overlapping validity interval for this creator (Bonus 2)')
    WHERE EXISTS (
        SELECT 1 FROM CreatorTierPeriod existing
        WHERE existing.creator_id = NEW.creator_id
        AND NEW.valid_from < COALESCE(existing.valid_to, '9999-12-31T23:59:59Z')
        AND COALESCE(NEW.valid_to, '9999-12-31T23:59:59Z') > existing.valid_from
    );
END;
-- (a matching trg_creatortier_no_overlap_update trigger excludes the
-- row being updated from its own overlap check via OLD.valid_from)
```

Verified with two real cases against the populated database, using creator 905 (existing period: valid_from = '2025-09-01T00:00:00Z'):

```
INSERT INTO CreatorTierPeriod (creator_id, tier, valid_from, valid_to)
VALUES (905, 'gold', '2025-09-01T00:00:00Z', NULL);
-- Runtime error: Overlapping validity interval for this creator (Bonus 2)
```

```
INSERT INTO CreatorTierPeriod (creator_id, tier, valid_from, valid_to)
VALUES (905, 'bronze', '2020-01-01T00:00:00Z', '2020-06-01T00:00:00Z');
-- succeeds, no error
```

The overlapping insert was correctly rejected; the genuinely non-overlapping insert for the same creator succeeded as expected.

What this version cannot guarantee that a declarative EXCLUDE constraint would: (1) it only fires through ordinary per-row INSERT/UPDATE — a corrupted or maliciously direct file write bypassing SQLite’s SQL layer entirely would not be caught, though no ordinary DML path exists for this; (2) it checks only the row being written, not the whole table’s consistency — if an overlap already existed before this trigger was created (it does not, in this dataset, since E.3’s generator built each creator’s periods sequentially with advancing `cursor_time`), the trigger would not retroactively detect or repair it; (3) under SQLite’s single-writer model (confirmed empirically in T3), no genuine race condition between two concurrent overlapping inserts can occur within one file the way it could under a true multi-writer MVCC engine, so this trigger is in one sense stricter than SQLite’s own concurrency model strictly requires — but would still be necessary in a hypothetical multi-writer deployment, or if SQLite’s file were ever accessed by more than one process without going through the single-writer serialization T3 demonstrated.

Bonus 3 — Enforcing the deletion grace period (not attempted, honestly explained)

This item is deliberately **not** implemented as a trigger, because triggers cannot do the job it asks for. A trigger fires on INSERT/UPDATE/DELETE — it has no mechanism to intercept or filter a SELECT. The requirement (“no query can see a suspended account”) is fundamentally a *read-time* filtering problem, which is exactly what PostgreSQL’s row-level security solves by transparently rewriting every query’s WHERE clause at the engine level for a given role. SQLite has no equivalent — no row-level security, no query rewriting, nothing that intercepts a SELECT before it runs.

The honest answer to “where does this enforcement actually live”: in this schema, it lives in `v_public_profile` (G.1), which filters to `account_state = 'active'` — but that protection covers only queries that go *through* that view. Any query written directly against `AppUser` — including every other view in `views.sql`, and any ad-hoc query a developer writes during debugging — has no such filter and would return a deactivated or pending-deletion account’s row in full, phone number and all, if asked. **What defeats this protection is precisely that: querying the base table directly instead of the view.** There is no declarative mechanism in SQLite that closes this gap; the only real fix is organisational discipline (a rule that no code path queries `AppUser` directly, enforced by code review or a linter, not by the database) or, if this were being built on PostgreSQL instead, an actual CREATE POLICY row-level security rule attached to `AppUser` itself, which would apply regardless of how the table is queried. Claiming a trigger-based workaround here would be presenting a partial, read-path-specific patch as a general solution it is not, which is why it is left undone rather than faked.

Appendix — LLM Assistant Usage

I used an LLM assistant (Claude) at a few points as a sounding board — mainly to check my understanding of SQLite-specific behaviour I wasn't fully sure of, and to get a second opinion on a couple of design choices before committing to them. Everything in this submission — the schema, the generator, the queries, the transaction scripts — is my own work, run and verified on my own machine. Below are five points where advice I got was wrong, and I caught it through my own testing rather than taking it at face value.

1. Advice on getting a power-law follower distribution didn't actually produce one

I asked how to make follower counts look power-law-shaped in my generator, since that's what the brief calls for. The suggestion was to skew how many accounts each user follows and pick targets at random. I coded this up, ran the generator, and checked the actual follower counts myself (`SELECT followee_id, COUNT(*)... ORDER BY n DESC`) rather than assuming it had worked — the top follower count came out to only 48 across 5,000 users, nowhere near power-law. I worked out why myself: follower count is *in*-degree, and skewing *out*-degree while sampling targets uniformly cancels the skew out completely. I fixed it by assigning each user a popularity weight and using that to bias who gets followed instead. After the fix: 44.9% of users have zero followers, median is 1, and the top 1% hold 55% of all follows — a real power-law shape.

2. A candidate-key claim I was given for D.1 turned out to be false

While working through the BCNF analysis for `ModelPricingPeriod`, I was told `{model_id, rate_type, valid_to}` could be treated as a second candidate key. Before including it, I checked it myself and found it doesn't hold: `valid_to` is `NULL` for the currently-active period, and a key can't contain a nullable column, since two different models' active rows would both show `valid_to = NULL` at the same time. I corrected my analysis to state only the one real candidate key, and added a note explaining why the second one fails, since it's a natural but wrong instinct worth flagging.

3. The suggested transaction-failure trick doesn't actually fail in SQLite

For T1, I initially tried the exact failure-injection approach the task sheet itself suggests as a template — `SELECT 1/0;`. I ran it before relying on it and got `NULL` back with no error: SQLite's integer division by zero is defined behaviour, not an exception, unlike PostgreSQL or MySQL. My transaction never actually failed, so my first attempt at this demonstration proved nothing. I swapped in a genuine `UNIQUE` constraint violation instead (a duplicate retraction for the same like), which I verified does fail properly.

4. I assumed a failed statement rolls back the whole transaction — it doesn't

Even after fixing #3, I checked the database state myself between the failing statement and my `ROLLBACK`, rather than assuming SQLite would auto-undo everything. It doesn't: the transaction was still open, and the earlier successful `INSERT` was still sitting there, uncommitted. If I'd trusted my first instinct — that a `COMMIT` placed after a failing statement simply wouldn't run — I'd have missed that an explicit `ROLLBACK` is actually required, since SQLite's default conflict handling only undoes the one statement that failed. I added the explicit `ROLLBACK` and documented the finding in `transactions.sql` so I wouldn't rely on behaviour SQLite doesn't actually provide.

5. My first attempt at the B.1 diagrams failed to compile, twice

While building the ER diagrams, I used multi-line TikZ node labels without realising they needed `align=center` set on the node style, which threw a `Not allowed in LR mode` error the first time I compiled. Separately, I later found one of my $\text{\LaTeX}$ sections had gotten cut off mid-edit, leaving an unclosed `\emph{}` block running straight into `\end{document}`. I only caught both by actually compiling and reading the error output rather than assuming the source was fine, and fixed each by restoring the missing option and the missing content, recompiling after each to confirm the PDF actually built.